

AetherGrid Operations & Support Runbook

Real-Time Microgrid Telemetry & Energy Load Balancing Full-Stack Architecture

VERSION 2.1 • SYSTEM CODE: AG-CORE-PRD • CLASSIFICATION: CRITICAL INFRASTRUCTURE

1. Architecture Topology & Service Inventory

AetherGrid processes real-time telemetry from thousands of smart commercial micro-inverters. The system stack is distributed across Kubernetes clusters running stateful ingestion pipelines, stream processors, and a React telemetry UI.

Component Name	Port / Protocol	Function Description	Probe Endpoint
aether-frontend	443 / HTTPS	React operator dashboard displaying real-time power grids	/healthz
aether-ingest-api	8080 / HTTP (gRPC)	High-throughput ingestion service accepting inverter telemetry	/metrics/ ping
stream-processor	9092 / TCP	Apache Kafka cluster handling real-time event routing	/status/ broker
timescale-db	5432 / PostgreSQL	Time-series database storing historical grid loads	SELECT 1;

2. Severity Levels & Incident Escalation SLAs

Operations engineers must classify any anomalous alert against the following framework:

- **Sev-A (Critical Outage):** Ingestion pipeline stoppage affecting > 15% of regional microgrids or complete database lockup. *SLA: Acknowledge < 10 mins.*
- **Sev-B (Degraded Performance):** Stream processing lag exceeding 45 seconds or intermittent frontend rendering failures. *SLA: Acknowledge < 30 mins.*
- **Sev-C (Minor Alert):** Individual node disk pressure warnings or non-critical log anomalies. *SLA: Acknowledge < 2 hours.*

3. Telemetry Stream Balancing Formulas

The processing lag threshold L_{lag} is determined by incoming packet frequency f , worker thread allocation W , and queue backlog Q :

$$L_{lag} = \frac{Q}{W \times f} + \epsilon$$

If L_{lag} exceeds 3.5 seconds, manual partition scaling is required.

4. Standard Diagnostic & Maintenance Procedures

4.1 Inspecting Ingestion Pod Health

To check the active state of the high-throughput ingestion microservices:

```
kubectl get pods -n aether-production -l app=aether-ingest-api
```

Tip: Always check pod restart counts before restarting manually. A high restart count usually indicates an out-of-memory (OOM) killer triggered by burst telemetry traffic.

4.2 Purging Stale Kafka Consumer Offsets

If stream processors hang following a network partition event:

- 1. Scale down the consumer deployment:

```
kubectl scale deployment aether-processor --replicas=0 -n aether-production
```

- 2. Reset consumer group offsets to latest committed partitions using the Kafka admin utility tool.
- 3. Scale the deployment back up:

```
kubectl scale deployment aether-processor --replicas=6 -n aether-production
```

Warning: Never execute a direct database truncation on the `timescale-db` hyper-table without taking an immediate snapshot backup first.

5. Escalation Contacts & On-Call Rotation

Escalate unresolved incidents to tier-2 engineering according to the schedule below:

On-Call Role	Slack Channel	Primary PagerDuty Policy
Infrastructure Reliability Lead	#sre-oncall	PD-Aether-Infra-Primary
Database Operations Specialist	#db-support	PD-Aether-DB-Escalation
Grid Engineering Director	#grid-leads	Sev-A Executive Escalation Only